QORA: Neural-Enhanced Interference-Aware Resource Provisioning for Serverless Computing

Ruifeng Ma, Yufeng Zhan, Chuge Wu, Zicong Hong, Yasir Ali, and Yuanqing Xia, *Fellow, IEEE*

***Abstract*— Serverless is an emerging cloud paradigm that offers fine-grained resource sharing through serverless functions. However, this resource sharing can cause interference, leading to performance degradation and QoS violations. Existing white box-based approaches for serverless resource provision often demand extensive expert knowledge, which is challenging to obtain due to the complexity of interference sources. This paper proposes QORA, a neural-enhanced interference-aware resource provisioning system for serverless computing. We model the resource provisioning of serverless functions as a novel combinatorial optimization problem, wherein the constraints on the queries per second are derived from neural network performance model. By leveraging neural networks to model the nonlinear performance fluctuations under various interference sources, our approach better captures the real-world behavior of serverless functions. To solve the formulated problem efficiently, rather than adopting commercial optimizer solvers like Gurobi, we propose a two-stage-VNS algorithm that searches discrete variables more efficiently and supports Sigmoid activations, avoiding introducing redundant discrete variables. Unlike pure machine learning methods lacking theoretical optimal guarantees, our approach is rigorously proven globally optimal based on optimization theory. We implement QORA on Kubernetes as a serverless system automating resource provisioning. Experimental results demonstrate that QORA reduces the QoS violation rate by 98% while reducing up to 35% resource costs compared with the state-of-the-arts.**

***Note to Practitioners*—From the perspective of cloud service providers, this paper considers the automatic resource provisioning for serverless functions. To improve hardware utilization, cloud providers tend to co-locate serverless functions on the same server. However, co-located functions compete for shared resources (memory bandwidth, L3 cache, etc.), which causes interference and leads to performance degradation and QoS violations. We use neural networks to build the performance models of interference-prone serverless functions and form the resource allocation optimization problem with neural network performance models as constraints. Compared to white box modeling methods, our neural network modeling adapts to complex and variable interference. Compared to deep reinforcement learning methods, our combinatorial optimization methods have stronger interpretability. In order to solve this optimization problem efficiently, we design the two-stage-VNS solution algorithm. We implement QORA on Kubernetes as a serverless system, which can automatically allocate computing resources. Experiments with small-scale real clusters and large-scale simulations demonstrate the effectiveness of QORA.**

***Index Terms*— Serverless computing, performance interference, resource provisioning.**

Received 19 June 2024; revised 15 October 2024; accepted 1 January 2025. Date of publication 15 January 2025; date of current version 4 April 2025. This article was recommended for publication by Associate Editor W. Beksi and Editor K. Liu upon evaluation of the reviewers' comments. This work was supported by the National Natural Science Foundation of China under Grant 61836001, Grant 62203049, Grant 62102022, and Grant 62473047. *(Corresponding authors: Yuanqing Xia; Yufeng Zhan.)*

Ruifeng Ma, Yufeng Zhan, and Chuge Wu are with the School of Automation, Beijing Institute of Technology, Beijing 100081, China (e-mail: ma.rui-feng@qq.com; yu-feng.zhan@bit.edu.cn; wucg@bit.edu.cn).

Zicong Hong is with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, SAR (e-mail: zicong.hong@connect.polyu.hk).

Yasir Ali is with Zhengzhou Research Institute, Beijing Institute of Technology, Zhengzhou 450000, China (e-mail: yasiralibit@gmail.com).

Yuanqing Xia is with the Zhongyuan University of Technology, Zhengzhou 450007, China, and also with the School of Automation, Beijing Institute of Technology, Beijing 100081, China (e-mail: xia_yuanqing@bit.edu.cn).

Digital Object Identifier 10.1109/TASE.2025.3526197

I. INTRODUCTION

SERVERLESS is an emerging cloud paradigm that enables developers to run code in the cloud without managing and operating cloud resources [1], [2], [3], where compute resources scale automatically and serverless functions run seamlessly. Serverless also offers a pay-as-you-use pricing model, where developers pay only for the execution time and resources consumed by their serverless functions rather than for idle or reserved capacity. Serverless platforms can quickly autoscale hundreds of function instances to support dynamic request loads.

On a serverless platform, the burden of operating multiple function instances is transferred from the user to the cloud provider. To improve hardware resource utilization, multiple function instances are deployed on the same server [4]. However, function instances deployed on the same physical server contend computing resources (eg. CPU, memory, memory bandwidth, L1, L2 and L3 caches, etc) [5], [6], [7] which leads to interference. According to our experiments in § III-B, interference can decrease the performance (i.e., queries per second, QPS) by up to 60%.

Some previous works have been proposed to reduce interference while improving hardware resource utilization. Firstly, some works [5], [6] try to isolate shared resources as finely as possible between serverless functions to eliminate interference. However, the Linux Kernel's isolation of shared resources (e.g., L3 cache and memory bandwidth) relies on tools like Intel resource director technology (RDT) [8] to isolate. Those tools only support a limited number of CPU models and cannot isolate other shared resources such as L1 and L2 cache. This limits their application scopes. Secondly, other studies [9], [10], [11], [12] analyze the interference characteristics between function instances and schedule non-affinity tasks to

1558-3783 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

different physical servers. However, modeling the interference between function instances introduces a large combination space and requires extensive prior expert knowledge. Therefore, interference between function instances remains an unsolved problem in serverless computing.

In this paper, we propose QORA to alleviate the effect of interference in serverless computing, i.e., increasing the hardware utilization to reduce resource costs while guaranteeing the quality of service (QoS) for serverless functions. However, two major challenges need to be addressed as follows. *(1)* Difficulty for performance modeling: Massive interference sources cause the serverless functions to present complex and variable nonlinear characteristics. This makes the presupposed explicit nonlinear expression modeling extremely difficult. *(2)* Resources provisioning problem is difficult to optimize: The resource provisioning problem is a mixed integer programming problem (MIP) because the number of function instances is an integer. In § V-B, the resource provisioning problem is proved to be NP-Hard. Resource provisioning on modern serverless platforms (e.g., Azure Functions [1] can host up to 200 instances.) is a large-scale optimization problem, which can reach 1.33×10^{36} configuration choice as shown in § III-B. Solving the resource provision problem with huge configuration spaces is complex and time-consuming.

For the first challenge, QORA employs neural networks to construct performance models for each serverless function which can finely fit performance variations caused by interference. Neural networks are proven better fit various nonlinear properties [13], [14]. In § III-B, our experiment demonstrates that it models the fluctuating serverless function's performance better than presupposed piece-wise fitting method [15]. The neural networks are added as performance constraints, ensuring the QPS that the serverless function can support is greater than the actual requested QPS. We then translate the neural network into the constraints and variables combinatorial optimization that can handle using open source software including Omlt [16] and Pyomo [17]. Essentially, QORA integrates the performance model, which has learned about interference, into the optimization problem's performance constraints, effectively addressing the interference issue. QORA primarily addresses resource interferences by throttling. Specifically, QORA mitigates resource interference by limiting the allocation of resources (e.g.: vCPU, memory, and number of instances), and scheduling serverless functions to servers with less shared resource contention.

For the second challenge, QORA devise a two-stage-variable neighborhood search (VNS) global convergent algorithm to solve the resource provisioning problem, which is rigorously proven globally optimal based on optimization theory in § V-C. Compared to commercial optimizer solvers like Gurobi which only supports Relu for non-linear activations [18], introducing redundant binary discrete variables, the devised two-stage-VNS algorithm searches discrete variables more efficiently and supports Sigmoid activations, avoiding introducing redundant discrete variables. We use the Pyomo block [17] to express and stack the Dense and Sigmoid layers of the neural network. Experiments have revealed that this algorithm reduces exhibits a solution speed that is 7.6 times faster than state-of-the-art. Besides this algorithm supports 10 times larger servers and decision variables than Gurobi. For configuring the number of instances on each server, which are the integer variables, we devise a novel global optimal VNS [19] algorithm. We use IPOPT [20] configuring the continuous variables including CPU and memory. IPOPT can apply gradient approximation and Jacobian approximation for neural network constraints, which global convergence is guaranteed by the interior point method with a filter algorithm [20]. We use the cost calculated by IPOPT as the objective value for the VNS algorithm.

Moreover, unlike pure machine learning methods (e.g., reinforcement learning based method [5], [21]) lack of optimal guarantee [22], QORA formulates the resource provision problem as a combinatorial optimization problem including neural networks as the performance constraint. The devised two-stage-VNS algorithm is proved as a globally optimal method in § V-C based on optimization theory.

The main contributions of this paper are as follows:

- We model the serverless function's performance with neural networks, which can adjust to performance variations due to various interference. And we formulate the resource provisioning problem including the neural networks as performance constraints. We prove this resource provisioning problem is an NP-Hard problem.
- We design a two-stage-VNS algorithm for the NP-Hard problem and illustrate the global convergence of the algorithm. Experiments have revealed that this algorithm exhibits a solution speed that is 7.6 times faster than state-of-the-art. Besides this algorithm supports 10 times larger servers and decision variables.
- We implement QORA, a serverless resource provisioning framework, and carried out experiments on real clusters to evaluate that QORA can decrease the QoS violation rate by 98% while reducing up to 35% resource costs compared with the state-of-the-arts.

The rest of this paper is organized as follows. § II discusses the related works. § III introduces the background and motivation. § IV presents the model design and problem formulation of the QORA. § V presents the QORA design details. The performance of the proposed method on various open-source benchmarks is evaluated by comparing it with the state-of-the-art in § VI. § VII concludes the paper.

II. RELATED WORK

To finely allocate resources, lots of work have been proposed. Firstly, some works [15], [23] model the performance of the serverless function using an explicit nonlinear fitting and then allocate resources according to the performance model. But explicit nonlinear fitting does not adapt the performance of serverless functions subject to uncertain interference, which leads to inaccurate resource allocation.

Secondly, some studies [5], [6] reduce interference by using tools such as Intel RDT to isolate shared resources (such as CPU L3 cache and memory bandwidth). However, those technologies often only support a limited number of CPU models and cannot isolate other shared resources such as L1 and L2 cache. Besides, the Linux Kernel's isolation of

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

shared resources relies on tools like Intel RDT. Thirdly, other works [5], [24] cap container resources to a fixed threshold. This is not an optimal configuration for both horizontal and vertical scaling due to the resource fungibility [6].

Moreover, Apostolopoulos et al. [25] propose an approach to offload social computing tasks on virtual machines and serverless computing. The authors model the problem as a Pure Nash Equilibrium and develop a low-complexity distributed algorithm to solve it. The model proposed by Apostolopoulos et al. [25] has a good performance by maximizing the user's expected prospect-theoretic utility.

Cilantro [26] is a performance-aware Resource Allocation based on an online learning mechanism that forms feedback loops with the jobs to estimate the resource to performance mappings and load shifts.However, Cilantro is limited to supporting only one type of resource, which can result in less efficient allocation. Additionally, it does not offer a detailed design for the utility function, which is needed for further work. Paragon [27] is an online and scalable scheduler that is aware of both heterogeneity and interference. Paragon utilizes existing information about previously seen workloads and collaborative filtering techniques, which can quickly and accurately classify unknown workloads based on their heterogeneity and interference characteristics across shared resources. However, Paragon employs a greedy strategy for scheduling multiple workloads, which can lead to sub-optimal resource scheduling. Additionally, it does not support resource allocation but only resource assignment. Quasar [28] employs similar collaborative filtering techniques as Paragon for classifying interference and server-type heterogeneity. Quasar considers both resource allocation and assignment, whereas Paragon focuses solely on resource assignment. However, Quasar only considers the performance target and does not take into account the cost target.AWARE [29] uses a meta-learning based approach to application and cluster environment modeling that improves model generalization and reduces training time. Besides, Shahrad et al. [30] develop a system that provides cluster-level support for LLC management in a public cloud.

Finally, some works [5], [31], [32] use reinforcement learning (RL) for resource allocation, and [33] uses a two-stage machine learning method. Similarly, Qiu et al. [34] design and implement a scalable and incremental multi-agent RL framework based on Proximal Policy Optimization (SIMPPO). Moreover, Autothrottle [35] is a bi-level resource management framework designed for microservices that operate under latency Service Level Objectives (SLOs). The framework separates the feedback loop of application SLOs from service resource control and connects them via performance targets. The core of Autothrottle [35] is an application-wide learning-based controller that is used to set performance targets, represented as CPU throttle ratios. These targets help individual per-service heuristic controllers achieve good resource allocation. However, machine learning (ML) algorithms rely on offline training and may fail to configure resources finely with entirely new types of cluster runtime. Therefore, interference between serverless function instances remains an unsolved problem in serverless computing.

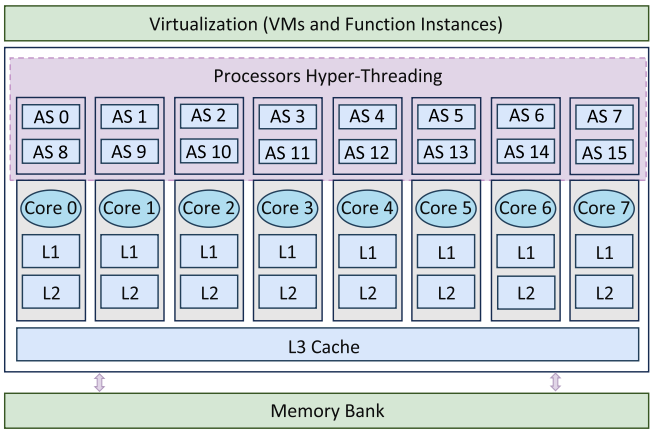


Fig. 1. Function instances contend for shared resources (e.g., memory bandwidth and caches).

In this paper, we propose QORA to alleviate the effect of interference in serverless computing, i.e., increasing the hardware utilization to reduce resource costs while guaranteeing the QoS for serverless functions.

III. BACKGROUND AND MOTIVATIONS

A. The Factors That Causes Interference

Interference between function instances is caused by shared resource contention [5], [6], processor time-multiplexing [36] and Hyper-Threading [37].

1) Multiple function instances compete for shared resources such as CPU L3 cache and memory bandwidth, and it is difficult to completely isolate serverless functions. Figure 1 illustrates the non-uniform memory access (NUMA) architecture of our experimental physical machine in § III-A, which is equipped with an 11th Gen Intel i7-11800H processor running at 2.30GHZ. This processor has eight cores, each with two hyper-threads, for a total of 16 hyper-threads and a single socket. Function instances contend for shared resources (e.g., memory bandwidth and caches). Linux containers [38] fall short when isolating shared resources, although it can allocate CPU cores and memory for each function instance. The Linux Kernel's isolation of shared resources requires tools like Intel RDT. But they often only support a limited number of CPU models and cannot isolate other shared resources such as L1 and L2 cache [39]. For instance, Intel RDT only provides partial functionality support for 12 types of CPUs and lacks support for other widely used Intel CPU models.

2) Technologies such as processor time-multiplexing [36] and Hyper-Threading [37] make multiple instances contend for CPU L1 and L2 caches, which can lead to performance degradation [7]. Therefore, there is still interference between function instances, and complete resource isolation is difficult.

We conducted experiments, as shown in Table I, on the physical machine to analyze the interference intensity of different types of resource contention on functions. We analyze the intensity of interference to serverless functions when contention occurs in L1 cache, L2 cache, L3 cache, and memory bandwidth. We tested three custom functions on the physical machine equipped with an Intel i7-11800H processor:

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

TABLE I
QPS DEGRADATION CAUSED BY RESOURCE INTERFERENCE

Interference Type	DenseNet	OCR	RF-Regression
L1 (Data Cache)	14.85%	26.16%	30.87%
L2 Cache	16.70%	26.59%	31.07%
L3 Cache	90.51%	96.06%	85.56%
Memory Bandwidth	81.15%	76.23%	66.64%

(1) Random forest regression (RF-Regression) [40] implemented by scikit-learn [41]; (2) Optical character recognition (OCR) implemented by Tesseract [42] ; (3) Densenet [43] image classification inference implemented by PyTorch's TorchVision. The chosen benchmark for generating resource contention is the containerized iBench [44], [45].

To create resource contention on L1 or L2 caches, 15 iBench workers are co-located on the same physical cores as the online service functions, but on different hyper-threads, to create interference. To create resource contention on L3 cache or memory bandwidth, online service functions are located on Arch State (AS) 0, corresponding to logical core 0 in Figure 1, 15 iBench workers are evenly distributed across logical cores 1-15 (AS 1-15), to create the interference and resource contention.

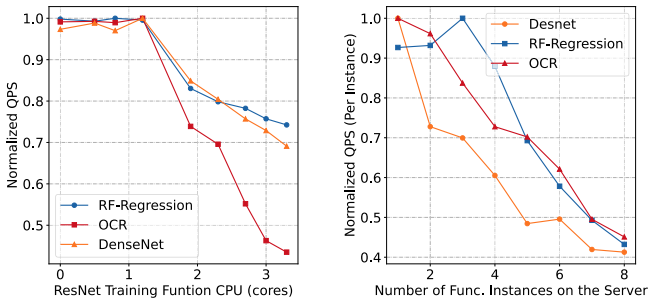
As shown in Table I, when L3 cache is contended, the three online serverless functions experience the most significant performance degradation. OCR experiences a drop in queries per second (QPS) of 96.06%, which means that it can only handle 3.96% of its original request volume. In contrast, the impact of contention on L1 (Data Cache) and L2 Cache is relatively minor, with DenseNet experiencing QPS reductions of 14.85% and 16.70% respectively. Regarding the impact of contention on memory bandwidth, RF-Regression experiences QPS reductions of 66.64%.

B. Motivations

Insight #1: *The diversified characteristics of interference make performance difficult to model.*

Two reasons cause the diversified characteristics of interference. Firstly, the massive types of interference sources cause diversified characteristics of interference. Secondly, the same interference source interferes differently with different serverless functions. We observe that interference results in more than 60% QPS decrease.

For the first reason, the various types of interference sources can be categorized as interference from *batch job functions* [46] and interference between *online service functions* [47]. The main difference between batch job functions and online service functions is the type of processing. Batch job functions run in the background on a schedule or trigger and are suitable for heavy data processing jobs [46]. Online service functions run on demand in response to events and are suitable for web applications or APIs [47]. They co-deployed on the physical server [4] to improve resource utilization but cause interference. We test the three custom functions in § III-A on a server equipped with 8 vCPU



(a) Performance degradation of online services functions caused by batch job functions

(b) Performance degradation caused by interference between online services functions

Fig. 2. The QoS of function degradation caused by interference.

AMD EPYC 7542 Processors. The QPS is shown as the max-normalization format, which is the formula as $\frac{x}{max(x)}$.

For the second reason, the same batch job function interferes differently with different online service functions. Figure 2(a) shows the interference of the Resnet training task (batch job function) on three different online service functions, including RF-Regression, OCR, and DenseNet inference.

We maintain the performance of online service functions, and batch job functions are considered to be the source of interference. In the experiment shown in Figure 2(a), we first deploy an RF-Regression function on the 8 vCPU server, limiting the CPU usage of the RF-regression function to 2 vCPU. We then add a ResNet training task and gradually increase its vCPU allocation on other vCPU cores. As the vCPU allocated to the ResNet training task increases, the performance of the RF-Regression gradually decreases because of interference. Next, we clear the 8 vCPU server to test the interference of the ResNet training task on the performance of the OCR and DenseNet inference functions. As the CPU consumption of the Resnet Training Task increases, the three online service functions are interfered with to varying degrees. As shown in Figure 2(a), when the Resnet training batch job function is set to 3.3 vCPU, the QPS of the OCR drops by more than 50% compared to no interference, while the performance of DenseNet and RF-Regression drops by 25-30%.

Moreover, interferences between online service function instances also show diversified characteristics. Figure 2(b) shows interferences between online service function instances. In the experiment illustrated in Figure 2(b), we start by deploying a single RF-Regression function instance with 0.82 vCPUs on the 8 vCPU server. The server runs the Ubuntu system and Kubernetes components like kubelet, leaving slightly fewer than 8 vCPUs available for use. To evaluate performance under near-full CPU utilization while deploying 8 instances, we can only limit the vCPU a little less than a full vCPU, which is 0.82. We then gradually increase the number of instances, which leads to an increase in the total QPS. However, due to interference among the online applications, as shown in Figure 2(b), the average QPS per instance decreases. The degree of this decrease indicates the level of interference among the RF-Regression instances. After this, we clear the 8 vCPU server to assess the interference patterns among

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

OCR and DenseNet inference function instances. As shown in Figure 2(b), the DenseNet and OCR functions' per-instance performance decreases immediately as the number of instances on the physical machine increases. However, the per-instance QPS of RF-Regression does not fall until the number of instances increases to 4.

The various interference characteristics make modeling performance by explicit nonlinear functions difficult because a variety of co-placement between serverless functions constitutes a huge combination space, which requires a lot of expert knowledge to measure and model every interference in the combination space. Moreover, since the configurable resources (CPU, memory, and the number of instances) are multi-dimensional, which makes high-order polynomial regression also needs to deal with a vast combination, and it is easy to fall into overfitting.

Insight #2: *Heterogeneity caused by interference makes optimization space grow exponentially.*

Interference results in heterogeneous server performance, which increases the optimization problem space and solution time. If there is no interference and the servers are isomorphic, the optimal configuration of CPU, memory, and the number of instances for a function would be identical on each server. This would reduce the decision space of the optimization problem. However, online service functions and batch job functions often share the same physical machine to improve hardware utilization [4], resulting in different interference levels. This makes the server performance heterogeneous, and the optimal configuration different on each server. The combination space becomes larger and more challenging to optimize.

Configurable resources of serverless functions include CPU, memory, and the number of instances. CPU and memory are continuous variables, which can be configured by IPOPT [20]. However, the number of instances is an integer, which is time-consuming for optimization. Assuming there are n machines, the instance number on each machine can be configured as $[0, 1, \ldots, m]$. Therefore, it has m^n types of configurations that need to be traversed.

The minimum vCPU for Azure Container is 0.25 [48]. We assume that the physical servers are equipped with 16 threads (vCPU) Intel E5-2667V3, then m would be $16/0.25 = 64$. Modern serverless platforms like Azure Functions [1] can host up to 200 instances across hundreds of servers. Assuming there are 20 optional servers, the configuration choice for instances is $64^{20} = 1.33 \times 10^{36}$, which is time-consuming to optimize. The huge search space requires us to design an efficient optimization algorithm.

Insight #3: *neural networks better describe the performance fluctuations under interference.*

Figure 3 shows the mean bsolute percentage error (MAPE) of neural network and Stepconfig's piece-wise fitting method [15] in fitting three serverless function performance models. The formula for calculating the MAPE is $MAPE = \frac{100\%}{n_{obs}} \sum_{t=1}^{n_{obs}} \left| \frac{x_{act} - x_{pre}}{x_{act}} \right|$, where x_{act} represents the actual value, x_{pre} represents the predicted value, and n_{obs} represents the total number of observations. The dataset for the neural networks is collected from a cluster, like the dataset described in § V-A, consisting of 87 to 135 discrete data points collected through a grid search. It aligns with the dataset collected during QORA's neural network online updates after interference changes. It can ensure the experiment accurately represent QORA's neural network performance during the online resource allocation.

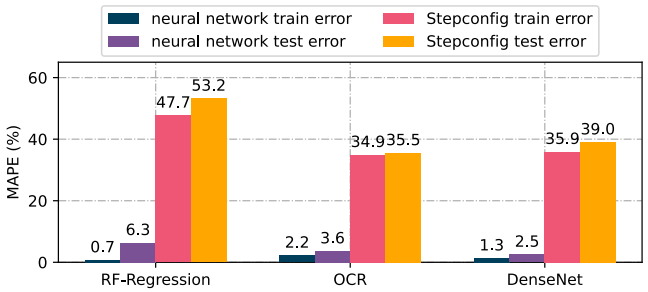


Fig. 3. Neural networks can better model performance under interference than explicit modeling.

The training error and test error of the neural network are both within 6.3%, while the error of the explicit fitting can exceed 50%. This means that the neural network fits the application performance under interference better than the explicit nonlinear fitting. The essential benefit of neural networks is that they have better generalization ability. The nonlinear regression models (i.e. Stepconfig [15]) assume a specific prior nonlinear pattern for serverless functions. When serverless functions are subject to interference and do not conform to the specific prior nonlinear pattern, significant modeling deviations occur. Moreover, new serverless functions may not conform to the specific prior nonlinear pattern. However, QORA's neural networks, with the multi-layer fully connected architecture, can not only fit the specific prior nonlinear pattern but also various nonlinear characteristics [13], [14], which show better generalization ability. Neural networks can capture complex patterns and trends [49], [50], particularly those variations created by interference and new serverless functions. This is the reason why neural networks outperform nonlinear regression models (i.e. Stepconfig [15]).

IV. Model Design and Problem Formulation

A. Neural Network Based Performance Model

We employ neural networks to construct performance models for each serverless function which can finely fit performance variations caused by interference as shown in § III-B For the same function instance, allocating the same CPU, memory, and other resources on different servers results in different QPS performance. Two reasons cause this performance difference. 1) As mentioned in § III, the performance degradation is caused by interference, including processor Hyperthreading or time multiplexing, and shared resource contention. 2) The performance difference caused by different types of hardware(e.g., CPU and memory). Therefore, assuming that we rent n servers and the serverless function can be deployed on the n servers, we build a resource-QPS model for each type of function on each server:

$$f_i(c_i, m_i, r_i) = q_i, \tag{1}$$

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

where $i \in [0, n]$ represents the server number. We assume that the multiple instances of the same function on the same server are homogeneous. c_i represents the number of vCPU allocated to each instance on server i, m_i represents the amount of memory allocated to each instance on server i, and r_i represents the number of instances of that type of serverless function on server i. q_i represents the QPS that all instances of that serverless function on server i can support. each serverless function on server i has a dedicated neural network performance model f_i. Since each serverless function has a different model f_i, the modeling process considers the different types of serverless functions, such as CPU-intensive tasks, memory-intensive processing, I/O-intensive tasks, or network-oriented activities.

Since the performance degradation is caused by the interference from other serverless functions mentioned in § III-B, the performance is various and challenging to model. Therefore, it is difficult to establish a "resource-QPS model" with explicit nonlinear fitting in the interference-prone cloud service scenario.

We train neural networks for each of the n servers to establish the resource-QPS model in Equation (1) for each type of serverless function. QORA's streamlined neural network is solely responsible for fitting the resource-QPS curve for each serverless function, without directly generating resource allocation actions. The neural network in QORA uses a 4-layer fully connected architecture, with each hidden layer containing 64 neurons. The neural network's streamlined structure is designed to ensure fitting performance as outlined in Insight #3 of § III-B while minimizing overhead as much as possible. Given the different nature of the resource QPS curves for different serverless functions, data collection and neural network modeling are performed independently for each function. In addition, as the INTERFERENCE changes, the resource-QPS curves are subject to change, requiring new data collection and model retraining. When interference changes, the retrained new neural network performance model will replace the original model. To reduce the retraining time, we use previous weights to initialize the neural network to accelerate convergence and reduce retraining time.

Specifically, we grid-like collect 87 to 135 discrete data points from the server in the real cluster. Each data point takes 4 seconds to collect, resulting in a total data collection time of up to 540 seconds. This relatively short time for comprehensive data gathering helps reduce the overhead typically associated with dataset collection. QORA initially allocates resources for a single instance based on the maximum available CPU and memory on the server. Then, QORA gradually reduces the CPU and memory allocation until it reaches the minimum resource configuration necessary for the serverless function to run. For the number of instances, once the CPU and memory values have been set, the function is tested with as many different configurations of the number of instances as possible, given the available CPU and memory on the server. This approach enabled us to observe how the function behaves under different load conditions, providing valuable insights into its performance. Additionally, we address scenarios where the number of serverless function instances can be scaled down to zero. Instead of testing in a real cluster, we generate 384 data points with QPS close to zero, but not zero, through code-based simulations. Since the QPS of a zero-instance function is naturally zero, these additional data points ensure that our neural network models are robust even when serverless functions scale down to zero instances. With these measures, we believe our method effectively balances the need for a comprehensive dataset to minimize data collection overhead.

TABLE II
MAIN NOTATIONS

c_i	vCPU allocated to each instance on the server i
m_i	memory allocated to each instance on the server i
r_i	the number of instances on the server i
n	the number of available servers
u_i^c	the available CPU on server i
u_i^m	the available memory on server i
l_i^c	function required minimum CPU
l_i^m	function required minimum memory
α	the request QPS
h	price for vCPU (Seconds)
g	price for Memory (GiB-Seconds)

Due to the compact data set and the streamlined structure of the neural network, the training time for 30,000 epochs is less than one minute. As a result, the total time for retraining and data collection is kept below 10 minutes. If the difference between monitored and predicted QPS exceeds the threshold due to other types of interference from other serverless functions on this server, QORA updates the corresponding performance neural network within 10 minutes by re-collecting data and re-training the network to eliminate the impact of inaccurate modeling as quickly as possible.

The neural network in QORA uses the Sigmoid activation function. Because if the Relu activation function is used, it would introduce a large number of 0-1 bool discrete variables, increasing the optimization problem's difficulty. Pyomo [17] and Omlt [16] are Python-based open-source optimization modeling language that provides a flexible and expressive syntax for formulating mathematical optimization problems, making it easier to define complex optimization models. We use open source packages Pyomo [17] and Omlt [16] to translate the neural network constraints to the variables and constraints that combinatorial optimization can handle. The number of continuous variables in the problem is $n \times 11$, the number of equality constraints is $n \times 9$, and the number of inequality constraints is $n \times 5 + 1$.

B. Problem Formulation

For each serverless function, we try to solve the main problem caused by interference: increasing the hardware utilization to reduce resource cost while guaranteeing the serverless function's QoS. To achieve that, we formulate a resource provisioning problem that minimizes resource costs while ensuring the QoS metric (i.e., QPS).

The decision variables of the problem are shown in the following matrix, with a total of $3 \times n$ variables, where n is

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

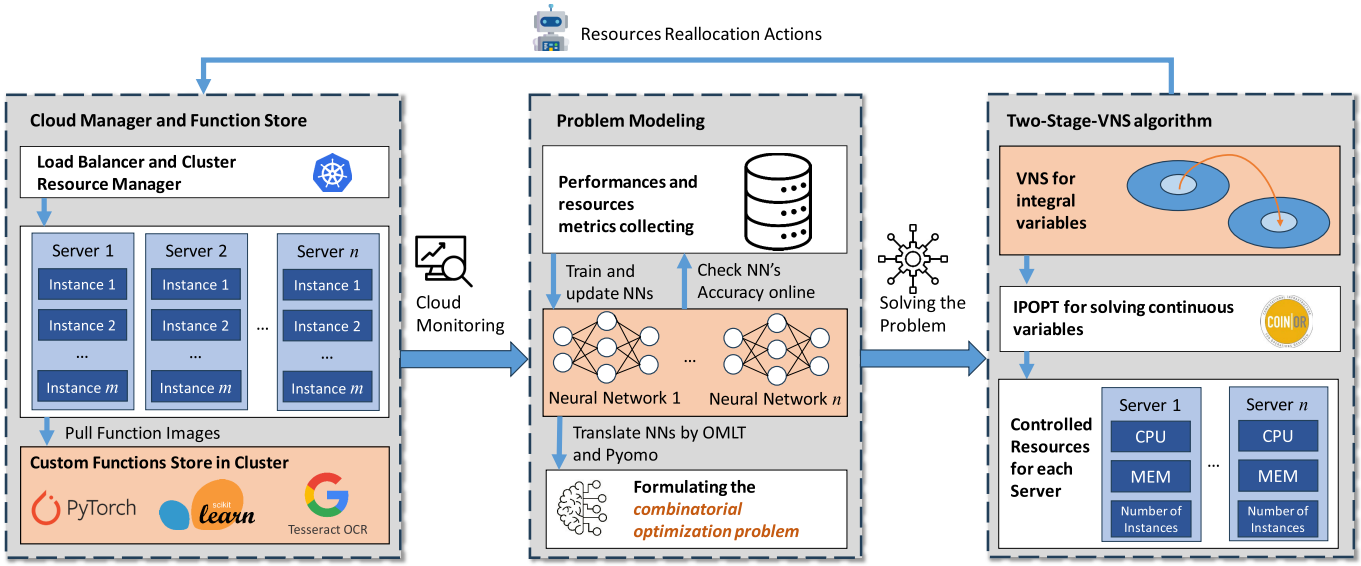


Fig. 4. The architecture of QORA.

the total number of machines. For $\forall i \in [1, \cdots, n]$, c_i and m_i are continuous variables representing the number of CPUs and memory, respectively, and r_i is an integer variable representing the number of instances.

$$\begin{bmatrix} c_1 & c_2 & \cdots & c_n \\ m_1 & m_2 & \cdots & m_n \\ r_1 & r_2 & \cdots & r_n \end{bmatrix}. \tag{2}$$

The optimization problem includes the following constraints. (3) represents the performance constraint, where the constant α is the request QPS must meet. And $f_i(c_i, m_i, r_i)$ is a neural network that characterizes the performance of the service,

$$\sum_{i=1}^{n} f_i(c_i, m_i, r_i) \geq \alpha. \tag{3}$$

Inequality (4) represents the resource constraint. Constants u_i^c, u_i^m represent the available CPU or memory on server i. Constants l_i^c, l_i^m represent the minimum CPU or memory required for the function to run steadily server i. For $\forall i \in [1, \cdots, n]$,

$$\begin{aligned} c_i &\geq l_i^c, \\ r_i \times c_i &\leq u_i^c, \\ m_i &\geq l_i^m, \\ r_i \times m_i &\leq u_i^m. \end{aligned} \tag{4}$$

The optimization problem can be formulated as:

$$\begin{aligned} &\min \left[\sum_{i=1}^{n} r_i \times (h \times c_i + g \times m_i) \right] \\ &Subject\ to: \quad (3), (4), \end{aligned} \tag{5}$$

where h and g are the prices of the resources, which are configured according to Azure Container Apps pricing [51].

V. QORA DESIGN

A. Overview

Figure 4 shows QORA's design framework. The users only have to specify the number of servers, the serverless function type, and the QPS needed. QORA pull images from the function store and allocate resources automatically. The users can also push their custom function images to the function store.

For problem modeling, QORA collects performances and resource metrics and trains neural networks for each type of serverless function to construct performance models that can adjust to performance variations due to interference. Then we translate the neural network into constraints and variables that are amenable to combinatorial optimization by OMLT [16] and Pyomo [17]. QORA checks the relative absolute value of the difference between the real performance (QPS) and the neural network calculated QPS of each server at every time stamp.

As shown in § III, other serverless functions in the cluster can cause performance interference. If the difference between monitored and predicted QPS exceeds the threshold due to interference source changing, QORA updates the corresponding performance neural network within 10 minutes by re-collecting data and re-training the network to eliminate the impact of inaccurate modeling as quickly as possible.

The devised two-stage-VNS algorithm ensures that the solution quality is optimal to solve the problem (5), which is an NP-Hard MIP problem.

In the first stage of the two-stage-VNS algorithm, we apply the VNS algorithm to determine the number of instances on each server. The number of instances is a discrete variable. We call the first stage the **N**umber of **I**nstance **C**onfiguration **P**roblem (**NICP**).

In the second stage, we use IPOPT to optimize the CPU and memory allocation for each instance. The configurations

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

of CPU and memory are continuous. IPOPT is an open-source software for nonlinear optimization, particularly effective for large-scale problems, using interior-point methods. The global convergence is guaranteed by the interior point method with a filter algorithm [20]. We use the cost function calculated by IPOPT as the objective value for the VNS algorithm in the first stage.

B. The Design of VNS Algorithm for NICP

In this section, we design the Algorithm 1 to solve the **NICP** and configure the number of instances on each server.

Theorem 1: **NICP** can be reduced to the Bounded Knapsack Problem [52], which is an NP-Hard problem.

Proof 1: We consider a linearized special case of **NICP**. The QPS increases linearly with the number of function instances. So $f_i(c_i, m_i, r_i)$ can be linearly approximated as $f_i'(c_i, m_i) \times r_i$. The problem can be re-formulated as follows:

$$\max\left[\sum_{i=1}^{n} r_i \times (-h \times c_i - g \times m_i)\right]$$
$$Subject\ to: \sum_{i=1}^{n} -f_i'(c_i, m_i) \times r_i \leq -\alpha. \tag{6}$$

We consider r_i as variables and c_i, m_i as constants. In polynomial time, (6) can be mapped into (7) with $A' = (-f_i'(c_i, m_i) \leftarrow w_i, (-h \times c_i - g \times m_i) \leftarrow v_i, \min(\frac{u_i^c}{c_i}, \frac{u_i^m}{m_i}) \leftarrow b_i, -\alpha \leftarrow W)$ [15],

$$\max\left[\sum_{i=1}^{n} v_i \times r_i\right]$$
$$Subject\ to: \sum_{i=1}^{n} w_i \times r_i \leq W \quad r_i \in \{0, 1, \ldots, b_i\}. \tag{7}$$

Since the negative profit and weight have no means in the standard knapsack problem, we map negative constants v_i, w_i, W into non-negative constants v_i', w_i', W' with (8):

$$\begin{aligned} v_i' &:= v_i - \min\{0, v_1, \ldots, v_N\} \\ w_i' &:= w_i - \min\{0, w_1, \ldots, w_N\} \\ W' &:= W - \min\{0, w_1, \ldots, w_N\} \times n. \end{aligned} \tag{8}$$

The (7) can be transformed into a bounded knapsack problem, which is an NP-Hard problem:

$$\max \sum_i v_i' \cdot r_i \sum_{i=1}^{n} w_i' \times r_i \leq W' \quad r_i \in \{0, 1, \ldots, b_i\}. \tag{9}$$

Note there is a case that $f_i(c_i, m_i, c_i)$ can be formulated as $f_i'(c_i, m_i) \times r_i$. So the **NICP** is an NP-hard problem.

Directly grid searching the **NICP** is inefficient, and traditional solutions to the knapsack problem cannot solve the nonlinear case of the **NICP**. Here, we design a VNS algorithm to solve the NP-hard **NICP**.

VNS consists mainly of two actions: "local search" and "shake", which alternate between different neighborhood structures to perform searches. Firstly, an initial solution and stopping criteria are selected during the initialization phase. Then, at each iteration, a "shaking" process randomly selects a neighborhood structure and generates a new solution from the current solution's specific neighborhood. This new solution is refined using a "local search" method to find the local optimum within that neighborhood. If the local optimum found is better than the current best solution, the algorithm "moves" to that point and continues the search using the new neighborhood structure. Otherwise, it switches to the next neighborhood structure. This process iterates continuously until the preset stopping criteria are met. Finally, the algorithm returns the best solution found as the solution to the problem.

Algorithm 1 VNS Algorithm for **NICP**

```
1  j ← 1
2  k ← maximum "shake" range
3  n ← the number of available servers
4  r ← the number of initial instances on each server
5  s ← initSolution(n, r)
6  while j ≤ MaxIterations do
7      Select a server i randomly from {1, 2, ..., n}
8      k' ← random integer from {2, 3, ..., k}
9      ϵ ← random integer from {−1, 1}
10     s' ← s
11     s'[i] ← s[i] + k' × ϵ
12     s'' ← s'
13     for a ← {1, ..., n} do
14         for b ← {1, −1} do
15             s_tmp ← s'
16             s_tmp[a] ← s'[a] + b
17             if s_tmp decreases the cost of IPOPT then
18                 s'' ← s_tmp
19     if cost(s'') < cost(s) then
20         s ← s''
21         j ← 1
22     else
23         j ← j + 1
24 Set function instances to each server as s
```

Specifically, in the Algorithm 1, Line 5 is to set an initial solution by configuring an equal number of instances on each server. In Line 7-11, QORA performs the "shake" in the VNS algorithm. QORA randomly choses $k' \in \{2, \ldots, k\}$ and randomly add or remove k' instances on selected server i. In practice, a larger value of k is set to achieve escaping from local optima and is set as 4. In Line 12-18, QORA performs a "local search" with $2 \times n$ iterations. In each search, QORA increases or decreases an instance on a certain server, and keeps the instance number configuration of other servers the same as s'. The configuration s'' that minimizes resource cost is selected. In Lines 19-21 of the algorithm, if the new solution s'' has a lower cost than the current solution s, QORA moves to the new solution. The Algorithm 1 repeats the "shake" and "local search" until the resource cost does not decrease during $MaxIterations$ iterations, which is set as 10. In practice, the VNS Algorithm for **NICP** is often superior to direct traversal.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

C. Global Convergence Analysis of VNS Algorithm for *NICP*

1) Analysis of Multistart Local Search Algorithm: Let us consider a simplified algorithm: Multistart Local Search Algorithm [19]. Here we replace "shake" with randomly selecting points multiple times from the solution space. The solution space is denoted as S. During the search process of **NICP**, the local search $\mathcal{L}$ transfers the initial action $s\prime$ to a locally optimal solution s''. Therefore, we can define a "trap" region for each local optimal solution L_j as follows:

$$A_j = \{x \in S \mid \mathcal{L}(x) \in L_j\}, j \in \{1, \ldots, N\}. \tag{10}$$

Therefore, by randomly selecting a point within the search space, the probability of it converging to the local optimal solution L_j is:

$$\gamma_j = P\{y \in A_j\}. \tag{11}$$

Generally speaking, the magnitude of γ_j is directly proportional to the size of the trapping region A_j.

Assuming there are N local minima. The number of integer variables in the solution space S is finite. Since $\gamma_j > 0$ and $\sum_{j=1}^{N} \gamma_j = 1$, we can calculate the probability θ_r of finding the rth smallest solution in terms of resource cost,

$$\theta_r = \sum_{i=1}^{r} \gamma_i. \tag{12}$$

The expected number of Multistart from the solution space is given by $E[Q_r] = \frac{1}{\theta_r}$. Let r to be equal to N.

2) Optimal Analysis of VNS Algorithm for NICP: For the variable neighborhood search algorithm for **NICP**, the neighborhood after the "shake" process is

$$\mathcal{N}(x) = \bigcup_{i=1}^{n} \mathcal{N}_i(x), \tag{13}$$

where n is the number of servers, $\mathcal{N}_i(x)$ refers to adding or removing k instances on the server i [19].

Theorem 2: If the value of k is sufficiently large, the VNS Algorithm for **NICP** is globally optimal convergent.

Proof 2: When k is sufficiently large, the neighborhood structure $\mathcal{N}(x)$ of the "shake" process can span multiple trap regions A_j until it iterates to a better solution. Therefore, any state that is not the global optimum is transient. We conclude that the sequence of the search process is globally convergent within a finite number of iterations.

In practical applications, the "shake" strategy of variable neighborhood search is often better than randomly selecting points from the solution space. Since the cost values returned to the Two-Stage-VNS algorithm by the IPOPT solver in the second phase are optimal [20], the value of each point explored by the VNS algorithm is reliable, ensuring that the algorithm has a guarantee of global optimality.

D. QORA Workflow

Firstly, Lines 1-5 complete the system initialization. In Line 1, the user submits the total number of servers. In Line 2, we set the list of servers that need to update the neural network (NN) model named $updatedNN$ as an empty list. In Line 3, QORA tests the QPS values under various resource configurations on each server in a grid-like manner, forming a dataset named $data$. In Line 4, QORA trains a neural network named $func$ to fit the performance function for each server. With the dataset, a total of n neural networks are trained. In Line 5, start the background process (Algorithm 2) for updating the performance function of the disturbed servers to eliminate the impact of inaccurate modeling as quickly as possible.

Secondly, in Line 8, the users set the reference QPS named α according to the current actual QPS at each time step. In Line 9, according to the resource provisioning algorithm in § V, QORA configure the CPU and memory size of each instance on each server and the number of instances. Note that QORA does not allocate function instances to the masked servers that need to update the Neural network model by checking the variable named $updatedNN$. In Line 10, QORA configure the resources for each server with the calculated resource configuration named $action$ and measure the real performance of the servers.

Finally, in Lines 11-14, QORA checks the relative absolute value of the difference between the real performance (QPS) and the neural network calculated QPS of each server i separately. In Line 14, if the difference is greater than or equal to the threshold, it is marked as a server that needs to update its neural network-based performance model to eliminate the impact of inaccurate modeling as quickly as possible. This difference may be caused by interference from other serverless functions as mentioned in § III-B.

Algorithm 2 QORA's Workflow

```
1  n ← num of servers
2  updatedNN ← empty list
3  data ← collect all server's performance
4  func ← train performance NN with data
5  backgroundUpdatingProcess.run()
6  # Main Process
7  while True do
8      α ← required QPS
9      action ← optimALGO(α, updatedNN, func)
10     relQPS ← allocateResources(action)
11     for each i ← {1, ..., n} do
12         a_i ← checkAccuracy(α, relQPS, i)
13         if a_i ≥ threshold then
14             updatedNN.append(i)
15 # Background Updating Process
16 while True do
17     n' ← num of elements in updatedNN
18     if n' not zero then
19         for i ← {1, ..., n'} do
20             data' ← collect Server i Perform.
21             func[i] ← train NN for Server i with data'
22             updatedNN.pop(i)
```

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

The background process communicates with the main process through the $updatedNN$ list and the performance model $func$, passing the list of servers that need to update the NN model. In the background process, QORA would update the neural network-based performance model for the marked servers. In Line 18, QORA first determines whether there are models that need to update the neural network-based performance model. In Line 20, QORA collects training data for these servers. In Line 21, QORA re-trains the neural network using the collected training data. In Line 22, QORA cancel the taint marks on these updated servers so that the optimization algorithm can schedule function instances to these servers.

VI. Performance Evaluation

A. Implementaion

Some other approaches such as Gunasekaran et al. [53] and Govind and González-Vélez [54] use Kubernetes to manage resources as the serverless platform. In a similar way to these approaches, we have built our serverless system directly on top of Kubernetes with more than 7000 lines of Python code. Directly invoking the Kubernetes API allows for a more convenient, transparent, and straightforward computational resource allocation for function instances, facilitating the creation of an automated serverless resource management system. QORA is running on the node of Control Plane on Kubernetes. We use NGINX as the load balancer and the Kubernetes Python Client to create user interaction clients and custom schedulers. The users only have to specify the number of servers, the serverless function type, and the QPS needed. QORA pull images from the serverless function store and allocate resources automatically. The users can also push their custom function's images to the serverless function store.

The cost of resources and the serverless function charges are calculated according to the Azure Container Apps pricing. QORA use the Kubernetes API to check the total amount of free resources (including CPU and memory) on each server. Then QORA configure the CPU and memory size of each function instance on each server and the total number of function instances. The workflow of the system is shown in Algorithm 2. We use OMLT [16] and Pyomo [17] to translate the neural network into variables and constraints that are easy to handle for optimization.

B. Experimental Settings

We use 3 servers that constitute a Kubernetes cluster with a single node of Control Plane. Each server is equipped with AMD EPYC 7542 Processors with (1) a frequency of 2.9GHz, (2) 8 cores, and (3) 16GB of memory. The cloud servers are connected with 4Gbps Ethernet cables. To verify the performance of the solution algorithm in a large-scale cluster, we build a simulated cluster with one hundred servers. To construct the simulated cluster, we collect the real serverless function data as a table from our serverless platform and provide QPS by looking up the table. We test the algorithm's performance in the simulator. To simulate the interference of batch job resource contention mentioned in § III-B, we set a random performance drop ranging from 3 to 30 percent for each simulated server.

TABLE III
CHARACTERISTICS OF THE WORKLOADS

App	Implement	QPS Range
RF-Regression	Scikit-learn	60-150
Densenet	Pytorch	12-60
OCR	Pytesseract	20-70

QORA uses QPS as the QoS indicator. The number of views of modern web applications (such as Wikipedia) changes periodically daily. Wikipedia provides the most granular time for page requests at the hour level [55], [56] and aggregates them by different access types (e.g., mobile-app or all-access). We collect one minute in the cluster to correspond to one hour of data in the real world. We proportionally scale the browsing data of Wikipedia pages and test the performance of QORA under aggregating types of mobile-app and all-access. The cost of resources and the serverless function charges are calculated according to the Azure Container Apps pricing [51]. The vCPU costs 0.000024 U.S. Dollars (USD) per second, and the memory costs 0.000003 per second per GiB. Function Apps cost 0.4 USD for one million requests processed.

Three popular online service functions have been used in our experiment to evaluate the performance of QORA.

Random Forest Regression (RF-Regression) [40]: The RF-Regression [40] implemented by scikit-learn [41], which QPS range is 60-150.

Densenet [43]: DenseNet is a convolutional neural network architecture for image recognition. It is implemented by PyTorch's TorchVision. The QPS range is 12-60.

OCR: Optical Character Recognition (OCR) defines the process of mechanically or electronically converting scanned images of handwritten, typed, or printed text. It is implemented by Tesseract [42]. Tesseract from 2006 until November 2018 was developed by Google. The QPS range is 20-70.

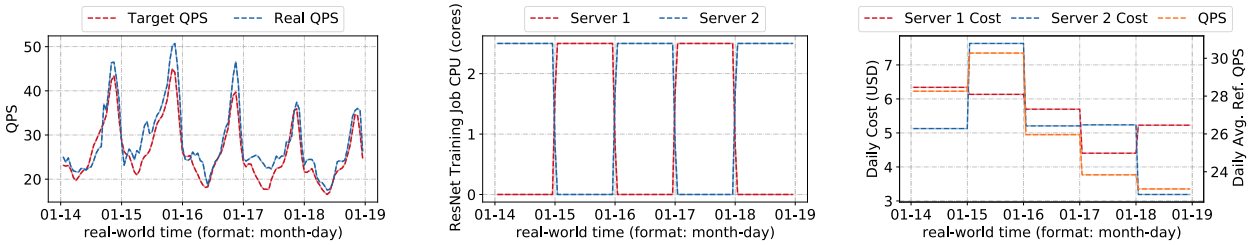
We implement the serverless function's API as an HTTP URL using Python's Flask. Then, we use the Apache HTTP server benchmarking tool to generate HTTP requests to the serverless function.

We simulate the external interference by running batch jobs. QORA only manages resources for online service functions and maintains the performance of online service functions. The interference source of the OCR function uses the batch job function of Resnet Training with 1.5 vCPU, while the interference source of the DenseNet and RF-Regession uses the batch job function of Resnet Training with 2.5 vCPU. We compare our proposed method with three state-of-the-arts.

StepConfig [15]: Stepconfig is a resource allocation method by simple performance modeling. Stepconfig employs a piecewise fitting method to model the performance of serverless functions. The piecewise fitting method can be seen as a simple performance modeling approach. It aims at minimizing resource costs while satisfying service level objectives (SLO).

Skynet [24]: It is an automated and adaptive approach to cloud resource management. Skynet estimates the resources

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.



(a) The QPS of DenseNet function (b) The CPU of batch job function (c) Daily resource cost and reference QPS

Fig. 5. QORA adapts to Wikipedia mobile-app request pattern for the DenseNet function.

required to achieve the target QoS and uses augmented Ziegler-Nichols [57] PID controllers to allocate the resources.

Autothrottle [35]: It is a bi-level resource management framework. The core of Autothrottle [35] is an application-wide reinforcement learning-based controller that is used to set performance targets, represented as CPU throttle ratios. These targets help individual per-service heuristic controllers achieve optimal resource allocation.

The neural network in QORA uses a 4-layer fully connected architecture, with each hidden layer containing 64 neurons. The neural network's streamlined structure is designed to ensure fitting performance as outlined in Insight #3 of § III-B, while minimizing overhead as much as possible. The action vector for each server $[CPU, Memory, Replica]$ is three-dimensional, so the input layer dimension is 3×64. The output is the number of queries per second (QPS) supported by the function instances on the server, and the output layer dimension is 64×1. The dimensions of the two hidden layers are both 64×64. The training process utilizes MSE Loss, with AdamW as the optimizer and a learning rate set to 0.005. QORA uses the Sigmoid activation function. Because if the Relu activation function is used, it will introduce a large number of 0-1 discrete variables. Regarding the choice of the Sigmoid activation function, when using the Omlt [16] open-source toolkit to transform the neural network's performance function into constraints and variables suitable for a combinatorial optimization problem, the use of piecewise activation functions (e.g., relu) would introduce a significant number of discrete variables, slowing down the optimization process. Among the smooth activation functions currently supported by Omlt, including Softplus, Sigmoid, and Tanh, we conduct experiments and select the one that performed best, which is the Sigmoid function. The $Maxiter$ in Algorithm 1 is set as 10.

C. Resource Costs Reduction in Real Cluster

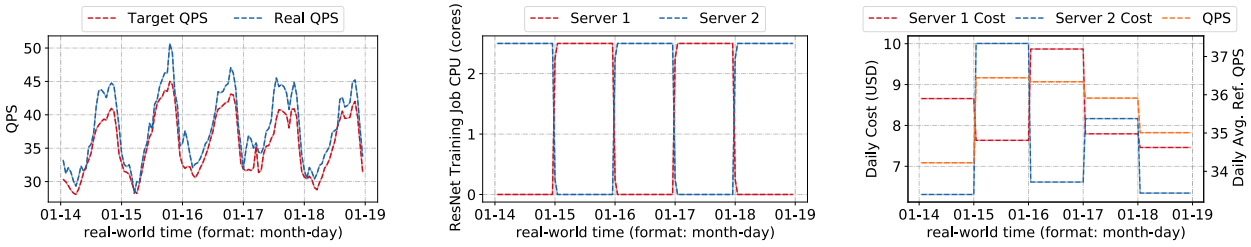
1) Qora Effectiveness: Figure 5(a) and Figure 6(a) show that QORA can meet Wikipedia's target QPS under both all-access or mobile-app request patterns. Figure 5 shows that QORA can optimize the DenseNet function's resource costs according to interference changes under Wikipedia mobile-app request pattern. We schedule the batch job alternately to assess QORA's ability to efficiently allocate resources to online service functions under frequent changes in interference. As the external interference source, the batch jobs actually consume 2.5 logical cores. As illustrated in Figure 5(b), between 0:00 on January 14 and 0:00 on January 15, Server 2 was interfered with by the batch job function while Server 1 was not interfered. Figure 5(c) shows the daily cost of DenseNet function, which is an online service function. From January 14 to January 15, because the resource cost of server 1 was higher than server 2, QORA allocated more resources to server 1 than to server 2. Actually, when the external source of interference is removed, the servers are homogeneous, with no differences. But when the running batch jobs interfere with the server, the performance of that server would degrade. Reducing the allocation of online service function resources to the server being interfered with can reduce the resource cost of the online service functions, which are managed by QORA. To mitigate the effect caused by interference, as shown in Figure 5(c), scQora allocated more resources to Server 1 and fewer resources to Server 2. Because we simulate the external interference by running batch jobs, reducing the allocation of online service function resources to the server being interfered with can reduce the cost of the serverless function's resources and alleviate interference.

Besides, Figure 6 shows QORA adapts to Wikipedia all-access request pattern for the DenseNet function. As illustrated in Figure 6(b), between 0:00 on January 15 and 0:00 on January 16, server 1 was interfered with by the batch job function while server 2 was not interfered with by the batch job. To mitigate the external interference caused by the batch job, as shown in Figure 6(c), scQora allocates more resources for DenseNet function to server 2 than to server 1, because the resource cost of server 2 was higher than server 1. Because we simulate the external interference by running batch jobs, reducing the allocation of online service function resources to the server being interfered with can reduce the cost of the serverless function's resources and alleviate interference.

Therefore, as shown in Figure 5 and Figure 6, scQora can alleviate interference by scheduling applications to servers with less shared resource contention.

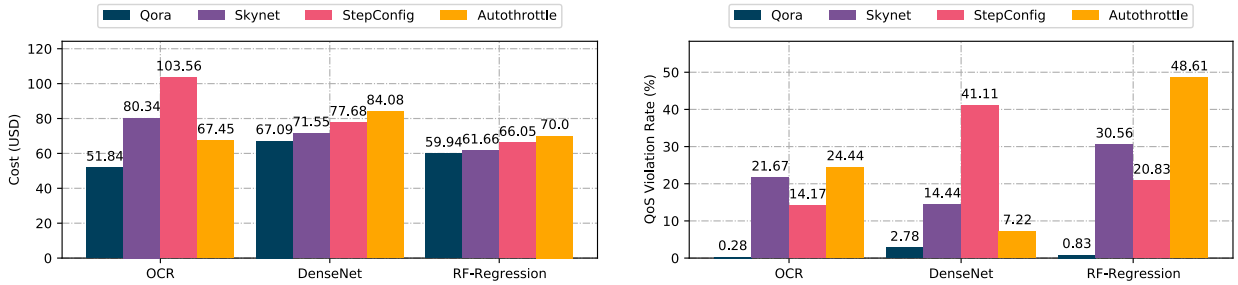
Stepconfig [15] is a resource allocation method by simple performance modeling. By comparing the experimental results of Stepconfig and QORA, we illustrate that the optimization overhead of QORA is less than the cost savings in serverless function resources. In the experiment in Figure 7, for the OCR function, The total cost for QORA optimization calculations is 0.15264 USD on node of Control Plane virtual machine

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.



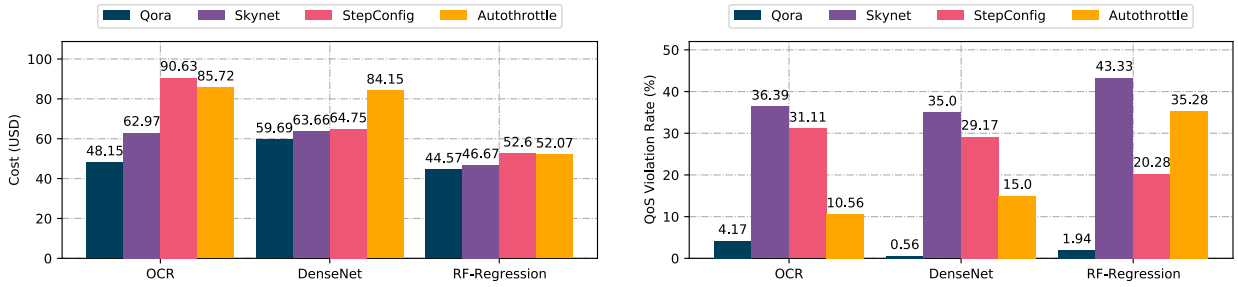
(a) The QPS of DenseNet function (b) The CPU of batch job function (c) Daily resource cost and reference QPS

Fig. 6. QORA adapts to Wikipedia all-access request pattern for the DenseNet function.



(a) The cost consumed by serverless functions (b) QoS violation rate

Fig. 7. Under Wikipedia all-access request pattern, evaluate four algorithms in real cluster during 5 real-world days.



(a) The cost consumed by serverless functions (b) QoS violation rate

Fig. 8. Under Wikipedia mobile-app request pattern, evaluate four algorithms in real cluster during 5 real-world days.

according to Azure container pricing [51]. Compared to Stepconfig, a resource allocation method with simple performance modeling, QORA saves 51.72 USD in resource costs on for the OCR function while reducing QoS violations by 98%.

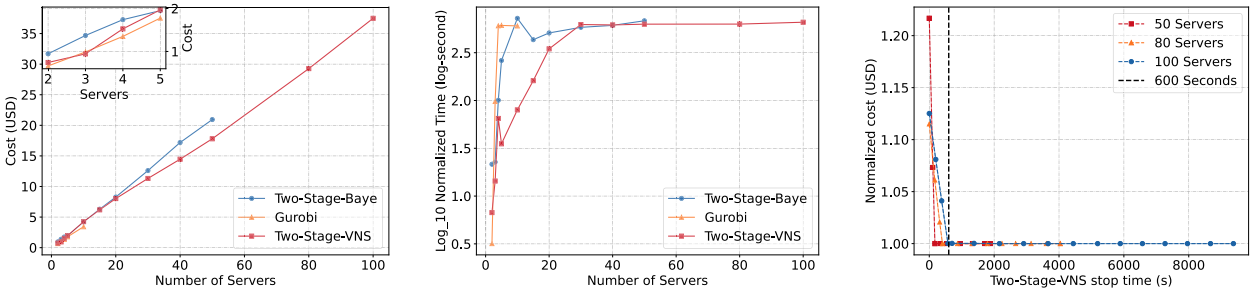
QORA's resource allocation optimization algorithm runs every a hour on a node of Control Plane virtual machine with 8 vCPUs and 16 GB of memory, taking about 5.3 seconds each time. The experiment corresponding to Figure 7 runs for 5 days and conducts 120 optimization calculations. According to Azure container pricing [51], the vCPU costs 0.000024 USD per second and the memory costs 0.000003 USD per second per GiB, making the cost of each QORA optimization calculation approximately 0.001272 USD. The total cost for QORA optimization calculations is 0.15264 USD for 5 days.

Figure 7 and Figure 8 show the QoS violation rate and resource costs during 5 real-world days. For OCR function under all-access request workload, QORA can decrease the QoS violation rate by 98% while reducing up to 35% resource costs compared with the state-of-the-arts.

2) Comparison With Skynet: Skynet [24] uses Ziegler–Nichols PID controllers, and it takes time to adjust the three gain parameters of proportional(P), integral(I), and differential(D). Since the PID controller is a SISO system, Skynet uses a set of PID controllers to control CPU and memory, respectively. When the application has QoS violations, the adaptive PID will increase the gain of all the PID controllers proportionally at the same time, thereby increasing the CPU and memory nearly proportionally at the same time.

Therefore, Skynet cannot achieve the optimal allocation of high-dimensional resources. This leads to the need of experts' experience to carefully set bigger initial gains for the PID which controls resources and has a higher impact on application performance [58]. For OCR function under

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.



(a) Resource cost (b) Time consuming (c) Early stopping

Fig. 9. Algorithms comparison and early stop evaluation.

all-access request workload, QORA can reduce up to 35% resource costs compared with Skynet.

3) Comparison With StepConfig: Stepconfig uses explicit nonlinear functions to model the performance of the serverless function. Stepconfig then poses and solves the resource optimization problem.

However, as described in § III-B, the explicit nonlinear expression used by Stepconfig to model the interfered serverless function has a large error. This model is used for the resource optimization problem. Therefore, the Stepconfig has a higher QoS violation rate. For OCR function under all-access request workload, QORA can decrease the QoS violation rate by 98% compared to StepConfig.

4) Comparison With Autothrottle: For RF-Regression function under mobile-app request workload, QORA can reduce up to 98.2% quality of service (QoS) violation rate and 14.4% cost compared with Autothrottle. This is because both QORA's neural network and Autothrottle's reinforcement learning model [35] would fail if the sources of interference changed and the cluster environment changed. However, QORA can detect changes in the environment by monitoring the difference between monitored and predicted QPS, allowing it to quickly update its streamlined neural network. In contrast, the reinforcement learning model of Autothrottle requires a longer update time.

The superior performance of QORA compared to Autothrottle [35] is related to modeling deviation arising from changes in interference sources within cloud clusters. Autothrottle's Tower employs contextual bandits, a lightweight form of online reinforcement learning (RL), to dynamically set performance targets that meet service level objectives (SLO) at the global level. However, when interference sources change, the information stored in the contextual bandits becomes outdated, leading to modeling deviation. It takes time to rectify the information stored in the contextual bandits. In contrast, QORA quickly identifies modeling deviation by comparing real QPS to model predicted QPS and quickly updates its streamlined neural network when modeling deviation is detected.

D. The Effectiveness in Large-Scale Simulations

To verify the effect of the QORA's two-stage-VNS algorithm, we test the algorithms' time-consuming and performance in the simulated cluster described in § VI-B.

To improve the solving speed of the algorithm, QORA implemented two optimizations: multi-process parallel search and an early stopping strategy. For the multi-process parallel search, QORA places each exploration point of the VNS local search into a process pool, setting the maximum number of processes in the pool to the number of CPU cores, and then conducting a parallel search. For the early stopping strategy, we set an early stop strategy where the algorithm begins to terminate after 600 seconds.

We set each server's expected QPS of the DenseNet function as 17, and the simulated cluster has n servers. The total QPS is $17 \times n$. The solution algorithm runs on a virtual machine server equipped with 64-vCPU AMD EPYC 7742 processors running at 2.25GHz with a single socket. There are two algorithms for comparison, one is Gurobi. Since Gurobi only supports the Relu activation, we replace the activation of the neural network from Sigmoid to Relu. The other is two-stage-Bayesian (two-stage-Baye) algorithm. We replace the discrete searching algorithm of two-stage-VNS from VNS with discrete Bayesian optimization. Figure 9(b) shows the three algorithms consumed time with logarithmic normalization to the base 10.

Figure 9 shows that the two-stage-VNS algorithm proposed in this paper is far better than Gurobi and two-stage-Baye algorithm on large-scale clusters (more than 30 servers), and on small-scale clusters, the performances of the three are close. When the cluster size is 10 servers, as shown in Figure 9(a), the resource costs of two-stage-VNS and Gurobi are similar. But as shown in Figure 9(b), after de-normalization, the solution time of two-stage-VNS is 79.8 seconds, while Gurobi's solution time is 603 seconds. QORA's two-stage-VNS exhibits a solution speed that is 7.6 times faster than state-of-the-art.

Within the time limit of 600 seconds, QORA's two-stage-VNS can support 10 times larger servers and decision variables. More specifically, the two-stage-VNS algorithm can support 100 servers, while Gurobi can support up to 10 servers. The two-stage-Baye algorithm only can support up to 50 servers. If the number of servers exceeds the limit, no feasible solution is returned within the limited time.

To analyze the impact of the early stop method on the cost increases, we conducted a supplementary experiment as shown in Figure 9(c). The horizontal axis represents the time at

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

which the algorithm is stopped, and the vertical axis indicates the normalized cost, calculated as $Normalized\ Cost = Cost/Min(Cost)$. $Normalized\ Cost$ can show the increment of cost relative to the minimum cost. As shown in Figure 9(c), For the clusters with 50 and 80 servers, early stopping does not lead to any increase in cost. The algorithm finds the optimal solution within 600 seconds. The time after 600 seconds is used to search other suboptimal neighborhoods to ensure the optimality of the current solution. For clusters with 100 servers, early stopping at 600 seconds results in only about a 0.02% increase in cost, which is acceptable. For a cluster of 100 servers, it can return an acceptable solution within 600 seconds. The supplementary experiments above demonstrate that the VNS algorithm is acceptable in an online system.

As shown in Figure 9(a), two-stage-VNS's resource cost increases with the total QPS ($17 \times n$) nearly linearly. This proves that the performance of the two-stage-VNS algorithm does not degrade as the cluster size increases.

VII. Conclusion

This paper is motivated by solving the main challenge caused by interference on serverless platforms: increasing the hardware utilization to reduce resource costs while QoS for serverless functions. In this paper, we propose QORA, a nearly globally optimal resource provisioning framework. QORA first formulates resource provisioning as a novel combinatorial optimization problem with neural networks as constraints. Then QORA apply a globally convergent two-stage-VNS algorithm, which exhibits a solution speed that is 7.6 times faster than state-of-the-art. Besides QORA supports 10 times larger servers and decision variables. Unlike pure machine learning methods (e.g., reinforcement learning based method [5]) lack of optimal guarantee [22], QORA is rigorously proven globally optimal based on optimization theory in § V-C. Experiments demonstrate that QORA can decrease the QoS violation rate by 98% while reducing up to 35% resource costs compared with the state-of-the-arts.

In the future, we intend to extend our work in the following ways: (1) When interference sources change, the neural network may experience modeling errors, necessitating retraining and causing computational overhead. In future research, methods like meta-learning [29] or transfer learning could be used to reduce this computational overhead. (2) Modern serverless architectures support serverless workflows [59]. Future research could consider resource allocation in serverless workflows. (3) Current serverless architectures also support GPUs. Future research could consider more resource types [60], such as GPUs, to enhance the applicability.

References

[1] Azure. (2023). *Azure Functions Hosting Options*. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-functions/functions-scale

[2] AWS. (2023). *What is AWS Lambda*. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

[3] Google. (2023). *Cloud Functions*. [Online]. Available: https://cloud.google.com/functions

[4] Q. Liu and Z. Yu, "The elasticity and plasticity in semi-containerized co-locating cloud workload: A view from Alibaba trace," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, Carlsbad, CA, USA, Oct. 2018, pp. 347–360.

[5] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in *Proc. 14th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*. Berkeley, CA, USA: USENIX Association, 2020, pp. 805–825.

[6] S. Chen, C. Delimitrou, and J. F. Martínez, "PARTIES: QoS-aware resource partitioning for multiple interactive services," in *Proc. 24th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, Providence, RI, USA, I. Bahar, M. Herlihy, E. Witchel, and A. R. Lebeck, Eds., Apr. 2019, pp. 107–120.

[7] VMware. (2023). *Hyperthreading*. [Online]. Available: https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.resmgmt.doc/GUID-FD71CBCA-E97C-4EFA-8A1B-32C09D5DF2A1.html

[8] Intel. (2023). *Intel-cmt-cat*. [Online]. Available: https://github.com/intel/intel-cmt-cat

[9] V. Gezer and A. Wagner, "Real-time edge framework (RTEF): Task scheduling and realisation," *J. Intell. Manuf.*, vol. 32, no. 8, pp. 2301–2317, Dec. 2021.

[10] P. Garefalakis, K. Karanasos, P. Pietzuch, A. Suresh, and S. Rao, "MEDEA: Scheduling of long running applications in shared production clusters," in *Proc. 13th EuroSys Conf.*, Porto, Portugal, R. Oliveira, P. Felber, and Y. C. Hu, Eds., Apr. 2018, pp. 4:1–4:13.

[11] R. Ma, Y. Zhan, T. Yan, Y. Xia, and Y. Ali, "Interless: Interference-aware deep resource prediction for serverless computing," in *Proc. 36th Chin. Control Decis. Conf. (CCDC)*, May 2024, pp. 3783–3788.

[12] L. Zhao, Y. Yang, Y. Li, X. Zhou, and K. Li, "Understanding, predicting and scheduling serverless workloads under partial interference," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, St. Louis, MO, USA, B. R. de Supinski, M. W. Hall, and T. Gamblin, Eds., Nov. 2021, pp. 1–14.

[13] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.

[14] V. Khezri, E. Yasari, M. Panahi, and A. Khosravi, "Hybrid artificial neural network–genetic algorithm-based technique to optimize a steady-state gas-to-liquids plant," *Ind. Eng. Chem. Res.*, vol. 59, no. 18, pp. 8674–8687, May 2020.

[15] Z. Wen, Y. Wang, and F. Liu, "StepConf: SLO-aware dynamic resource configuration for serverless function workflows," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, London, U.K., May 2022, pp. 1868–1877.

[16] F. Ceccon et al., "OMLT: Optimization & machine learning toolkit," *J. Mach. Learn. Res.*, vol. 23, pp. 349:1–349:8, May 2022.

[17] M. L. Bynum et al., *Pyomo—Optimization Modeling in Python*, vol. 67, 3rd ed., Cham, Switzerland: Springer, 2021.

[18] Gurobi. (2023). *Supported Regression Models*. [Online]. Available: https://gurobi-machinelearning.readthedocs.io/en/stable/mlm-supported.html

[19] J. Brimberg, P. Hansen, and N. Mladenovic, "Convergence of variable neighborhood search," *Les Cahiers du GERAD ISSN*, vol. 711, p. 2440, Mar. 2000.

[20] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Math. Program.*, vol. 106, no. 1, pp. 25–57, 2006.

[21] T. Qi, Y. Zhan, P. Li, J. Guo, and Y. Xia, "Hwamei: A learning-based synchronization scheme for hierarchical federated learning," in *Proc. IEEE 43rd Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2023, pp. 534–544.

[22] H. Xu, X. Zhan, and X. Zhu, "Constraints penalized Q-learning for safe offline reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, Washington, DC, USA. AAAI Press, 2022, pp. 8753–8760.

[23] J. Liu, S. Zhang, Q. Wang, and J. Wei, "Coordinating fast concurrency adapting with autoscaling for SLO-oriented web applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 3349–3362, Dec. 2022.

[24] Y. Sfakianakis, M. Marazakis, and A. Bilas, "Skynet: Performance-driven resource management for dynamic workloads," in *Proc. IEEE 14th Int. Conf. Cloud Comput. (CLOUD)*, Chicago, IL, USA, C. A. Ardagna et al., Eds., Sep. 2021, pp. 527–539.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.

[25] P. A. Apostolopoulos, E. E. Tsiropoulou, and S. Papavassiliou, "Risk-aware social cloud computing based on serverless computing model," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.

[26] R. Bhardwaj et al., "Cilantro: Performance-aware resource allocation for general objectives via online feedback," in *Proc. 17th USENIX Symp. Operating Syst. Design Implement. (OSDI)*. Boston, MA, USA: USENIX Association, Jul. 2023, pp. 623–643. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/bhardwaj

[27] C. Delimitrou and C. Kozyrakis, "Paragon: QoS-aware scheduling for heterogeneous datacenters," in *Proc. 18th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, vol. 48, New York, NY, USA, 2013, pp. 77–88.

[28] C. Delimitrou and C. Kozyrakis, "Quasar: Resource-efficient and QoS-aware cluster management," in *Proc. 19th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, New York, NY, USA, 2014, pp. 127–144.

[29] H. Qiu et al., "AWARE: Automate workload autoscaling with reinforcement learning in production cloud systems," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*. Boston, MA, USA: USENIX Association, Jul. 2023, pp. 387–402.

[30] M. Shahrad, S. Elnikety, and R. Bianchini, "Provisioning differentiated last-level cache allocations to VMs in public clouds," in *Proc. ACM Symp. Cloud Comput.*, New York, NY, USA, Nov. 2021, pp. 319–334.

[31] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Automatic scaling for microservices with an online learning approach," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Milan, Italy, E. Bertino, C. K. Chang, P. Chen, E. Damiani, M. Goul, and K. Oyama, Eds., Jul. 2019, pp. 68–75.

[32] F. Rossi, M. Nardelli, and V. Cardellini, "Horizontal and vertical scaling of container-based applications using reinforcement learning," in *Proc. IEEE 12th Int. Conf. Cloud Comput. (CLOUD)*, Milan, Italy, E. Bertino, C. K. Chang, P. Chen, E. Damiani, M. Goul, and K. Oyama, Eds., Jul. 2019, pp. 329–338.

[33] A. Horn, H. M. Fard, and F. Wolf, "Multi-objective hybrid autoscaling of microservices in Kubernetes clusters," in *Proc. 28th Int. Conf. Parallel Distrib. Comput.*, in Lecture Notes in Computer Science, vol. 13440, Glasgow, U.K., J. Cano and P. Trinder, Eds., Cham, Switzerland: Springer, Jan. 2022, pp. 233–250.

[34] H. Qiu et al., "SIMPPO: A scalable and incremental online learning framework for serverless resource management," in *Proc. 13th Symp. Cloud Comput.*, New York, NY, USA, Nov. 2022, pp. 306–322.

[35] Z. Wang, L. Ping-He, C.-J. M. Liang, F. Wu, and F. Y. Yan, "Autothrottle: A practical bi-level approach to resource management for SLO-targeted microservices," in *Proc. NSDI (USENIX Symp. Netw. Syst. Design Implement.)*, Jan. 2022, pp. 149–165.

[36] E. Caspi, R. Huang, Y. Markovskiy, J. Yeh, J. Wawrzynek, and A. DeHon, "A streaming multi-threaded model," in *Proc. 3rd Workshop Media Stream Processors (MSP)*, 2001, pp. 21–28.

[37] D. T. Marr et al., "Hyper-threading technology architecture and microarchitecture," *Intel Technol. J.*, vol. 6, pp. 1–12, Jan. 2002.

[38] (2023). *What's LXC?*. [Online]. Available: https://linuxcontainers.org/lxc/introduction/

[39] Cloud. (2023). *Resource Isolation Based on the L3 Cache and MBA*. [Online]. Available: https://www.alibabacloud.com/help/en/ack/ack-managed-and-ack-dedicated/user-guide/resource-isolation-based-on-the-l3-cache-and-mba

[40] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, Oct. 2001.

[41] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.

[42] Tesseract OCR. (2023). *Tesseract*. [Online]. Available: https://github.com/tesseract-ocr/tesseract

[43] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 2261–2269.

[44] C. Delimitrou and C. Kozyrakis, "IBench: Quantifying interference for datacenter applications," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Sep. 2013, pp. 23–33.

[45] Stanford-Mast. (2024). *IBench*. [Online]. Available: https://github.com/stanford-mast/iBench

[46] Azure. (2023). *Tutorial: Trigger a Batch Job Using Azure Functions*. [Online]. Available: https://learn.microsoft.com/en-us/azure/batch/tutorial-batch-functions

[47] Azure. (2023). *Choose the Right Integration and Automation Services in Azure*. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-functions/functions-compare-logic-apps-ms-flow-webjobs

[48] Azure. (2023). *Containers in Azure Container Apps*. [Online]. Available: https://learn.microsoft.com/en-us/azure/container-apps/containers

[49] M. S. Advani, A. M. Saxe, and H. Sompolinsky, "High-dimensional dynamics of generalization error in neural networks," *Neural Netw.*, vol. 132, pp. 428–446, Dec. 2020.

[50] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning (still) requires rethinking generalization," *Commun. ACM*, vol. 64, no. 3, pp. 107–115, 2021.

[51] Azure. (2023). *Azure Container Apps Pricing*. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/container-apps/

[52] D. Pisinger, "A minimal algorithm for the bounded knapsack problem," *INFORMS J. Comput.*, vol. 12, no. 1, pp. 75–82, Feb. 2000.

[53] J. R. Gunasekaran, P. Thinakaran, N. C. Nachiappan, M. T. Kandemir, and C. R. Das, "Fifer: Tackling resource underutilization in the serverless era," in *Proc. 21st Int. Middleware Conf.*, New York, NY, USA, Dec. 2020, pp. 280–295.

[54] H. Govind and H. González–Vélez, "Benchmarking serverless workloads on Kubernetes," in *Proc. IEEE/ACM 21st Int. Symp. Cluster, Cloud Internet Comput. (CCGrid)*, May 2021, pp. 704–712.

[55] Wikitech. (2023). *Analytics/AQS/legacy Pagecounts*. [Online]. Available: https://wikitech.wikimedia.org/wiki/Analytics/AQS/Legacy_Pagecounts

[56] Wikitech. (2023). *Analytics/Data Lake/Traffic/Virtualpageview Hourly*. [Online]. Available: https://wikitech.wikimedia.org/wiki/Analytics/Data_Lake/Traffic/Virtualpageview_hourly

[57] C. C. Hang, K. J. Åström, and W. K. Ho, "Refinements of the Ziegler–Nichols tuning formula," *IEE Proc. D Control Theory Appl.*, vol. 138, no. 2, pp. 111–118, 1991.

[58] R. Ma, Y. Zhan, Y. Xia, C. Wu, L. Yang, and R. Gao, "Sonnet: A control-theoretic approach for resource allocation in cluster management," *Future Gener. Comput. Syst.*, vol. 153, pp. 169–181, Apr. 2024.

[59] A. Arjona, P. G. López, J. Sampé, A. Slominski, and L. Villard, "Triggerflow: Trigger-based orchestration of serverless workflows," *Future Gener. Comput. Syst.*, vol. 124, pp. 215–229, Nov. 2021.

[60] H. Peng, Y. Zhan, D.-H. Zhai, X. Zhang, and Y. Xia, "Egret: Reinforcement mechanism for sequential computation offloading in edge computing," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3541–3554, Jun. 2024.



Ruifeng Ma received the B.S. degree from Shandong University in 2021 and the M.S. degree from Beijing Institute of Technology in June 2024. His research interests include cloud native technology, cloud resource management, and machine learning operations.



Yufeng Zhan received the Ph.D. degree from Beijing Institute of Technology (BIT), Beijing, China, in 2018. He is currently an Associate Professor with the School of Automation, BIT. Prior to join BIT, he was a Post-Doctoral Fellow with the Department of Computing, The Hong Kong Polytechnic University. His research interests include cloud/edge computing, predictive control, and machine learning systems.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.



Chuge Wu received the Ph.D. degree in control theory and control engineering from Tsinghua University, Beijing, China, in 2021. Since 2021, she has been with the School of Automation, Beijing Institute of Technology, Beijing, China, where she is currently an Associate Research Fellow. She has published over 20 papers. Her research interests include intelligent optimization and scheduling, modeling and algorithm design of computational system scheduling optimization problems.



Zicong Hong received the B.Eng. degree in software engineering from the School of Data and Computer Science, Sun Yat-sen University, Guangzhou, China. He is currently pursuing the Ph.D. degree with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong. His research interests include blockchain, edge/cloud computing, and federated learning.



Yasir Ali received the B.Sc. degree in electrical engineering from the University of Engineering and Technology at Peshawar, Peshawar, Pakistan, in 2014, and the M.S. degree in control science and engineering from the School of Automation, Beijing Institute of Technology, China, in 2017. He is currently a Researcher with Zhengzhou Research Institute, Beijing Institute of Technology. His current research interests include the security of cloud control systems and UAVs. He received the Best Paper Award 2019 from *Control Theory and Technology* (Springer).



Yuanqing Xia (Fellow, IEEE) received the Ph.D. degree in control theory and control engineering from Beihang University (previously known as Beijing University of Aeronautics and Astronautics), Beijing, China, in 2001. He was a Research Fellow in several academic institutions, from 2002 to 2008, including the National University of Singapore and the University of Glamorgan, U.K. Since 2004, he has been with Beijing Institute of Technology, China, where he is currently a Full Professor. Since October 2023, he has also been the President of Zhongyuan University of Technology. His research interests include cloud control systems, networked control systems, robust control and signal processing, active disturbance rejection control, and flight control. He is the Director of the specialized committee on cloud control and decision of Chinese Institute of Command and Control (CICC), a member of the 8th Disciplinary Review Group of the Academic Degrees Committee of the State Council and the Big Data Expert Committee of Chinese Computer Society, and the Vice Chairperson of the Internet of Things Working Committee of Chinese Institute of Instrumentation. He is the Deputy Editor of the *Journal of Beijing Institute of Technology* and an Associate Editor of *Acta Automatica Sinica*, *International Journal of Automation and Computing*, *Gyroscopy and Navigation*, and *IET Control Theory and Applications*.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:30 UTC from IEEE Xplore. Restrictions apply.